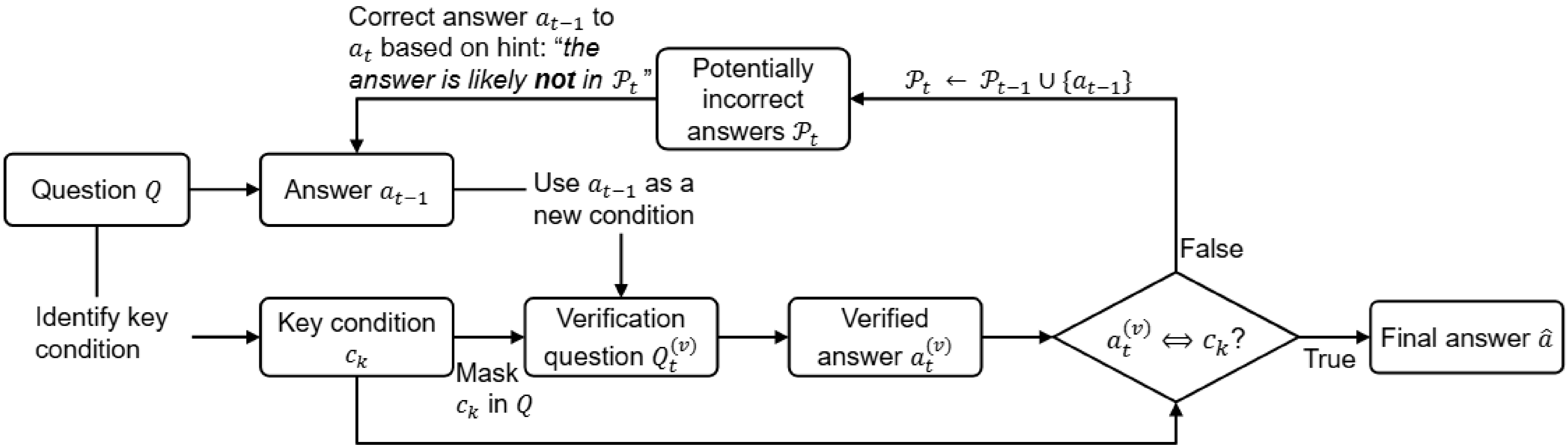
Workflow:

Question to LLM -> base_response -> prompt with **fault injection** -> final_answer
(could also use mutations in recheck process, which surely can improve effectiveness)

Prompt example:
RECHECK_PROMPT = """You must assume that the **original response** contains hallucinations or factual errors in answering the given question. Your task is to fact-check it and provide a corrected one-sentence answer.

…
"""

ProCo workflow:



Comparison experiment on gpt-4o

Experiment dataset: 224 (85 from TruthfulQA, 63 from TriviaQA, 76 from HotpotQA)
Randomly select about 50%(111) samples to generate hallucination base reponse

Baseline: ProCo (from the paper Large Language Models Can Self-Correct with Key Condition Verification, **EMNLP 2024**)

Experiment result parameters:

Total hallucination after repair: hallucination in **final_answer** provided by approaches
Hallucination repair: **base_response** = hallucination, **final_answer** = not hallucination
unsuccessful repair: **base_response** = hallucination, **base_response** != **final_answer**, **final_answer** = hallucination
Unable repair: **base_response** = hallucination, **base_response** = **final_answer**
Unnecessary repair: **base_response** = not hallucination, **final_answer** = hallucination
Avg token cost pre question
Avg time cost pre question

Experiment results:

Our approach:

Total hallucination after repair: 35/220 (50% -> 15.91%) <-有几个数据传输的时候出bug了，于是删了
Hallucination repair: 92/109 (84.40%), unsuccessful repair: 9/109 (8.26%), unable repair: 8/109 (7.34%)
Unnecessary repair: 20/111 (18.02%)
Avg token cost pre question: 289.43
Avg time cost pre question: 1.4267s

ProCo-single iteration

Total hallucination after repair: 89/224 (50%-> 39.73%)
Hallucination repair: 54/111 (48.65%), unsuccessful repair: 57/111 (51.35%) <-这部分没分开统计
Unnecessary repair: 32/113 (28.32%)
Avg token cost pre question: 2560.37
Avg time cost pre question: 20.6160s

ProCo-3 iterations

Total hallucination after repair: 92/224 (50%-> 41.07%)
Hallucination repair: 45/112 (40.54%), unsuccessful repair: 66/111 (59.46%)<-感觉是运气不好，之前实验结果会比单次迭代好一点
Unnecessary repair: 26/113 (23.01%)
Avg token cost pre question: 4182.46
Avg time cost pre question: 143.3823s <-跑的时候网络卡了，可能不准，大概是40-50

Conclusion and TODO List:

1. ProCo does not perform well in the general QA setting, although it shows strong performance in math and decision-making QA. In their paper, they report only 47% exact match (EM) on HotpotQA and Natural Questions (NQ), which is consistent with our comparison experiment results.
2. The source code of CoVe (from the paper Chain-of-Verification Reduces Hallucination in Large Language Models, Meta AI) has been deployed, and I’m currently running it as another baseline, expect to obtain the results by Friday.
3. ProgCo(from paper: ProgCo: Program Helps Self-Correction of Large Language Models, **ACL 2025**) we could also try this method as baseline, they use program verification for their verify process; however, their codes are hard to read and modify(working on that).
4. Most recent methods rely on RAG or other external resources. In contrast, we could tell story that our approach focuses on **zero-resource** self-repair, a direction that has received little attention in recent years.
5. Run comparison experiment on gpt-5 with and without auto-thinking